Parallelism Strategies with vLLM on RunPod

Parallelism Strategies with vLLM on RunPod

The Story

You are an ML engineer at a startup building an AI coding assistant. Your journey follows a common trajectory: you start with a single-GPU prototype and hit walls - latency walls, memory walls, throughput walls - and each parallelism strategy is the tool you reach for to break through.

Act 1 - The Prototype (Single GPU). You deploy a 7B model on one GPU. It works, but response times are sluggish for real-time chat.

Act 2 - The Latency Wall (Tensor Parallelism). Users complain about slow first-token times. You shard the model across GPUs to speed up computation per layer.

Act 3 - The Memory Wall (Pipeline Parallelism). Product wants to upgrade to a 70B model. It does not fit on a single node. You split the models layers across two RunPod machines.

Act 4 - The Throughput Wall (Data Parallelism). You launch publicly and get 100 concurrent users. One model replica cannot keep up. You replicate the model across GPUs to handle the load.

Act 5 - Production (TP x PP x DP). You combine all three strategies to serve a large model at scale.

Each act introduces a parallelism strategy, shows the exact commands to run it, and measures the impact using standardized latency and throughput metrics.

Metrics Glossary

Before diving in, here are the four metrics we will track throughout this tutorial. Understanding these is essential to evaluating the impact of each parallelism strategy.

TTFT - Time to First Token (ms)
The time from when the request is received to when the first token of the response is generated. This corresponds to the prefill (prompt processing) phase. TTFT directly controls perceived responsiveness - it is the thinking delay the user sees before output starts streaming.

TPOT - Time Per Output Token (ms)
The average time to generate each output token after the first. Calculated as . TPOT reflects the average decode speed. Lower TPOT means faster overall generation.

ITL - Inter-Token Latency (ms)
The time between consecutive token emissions during streaming. While TPOT is an average, ITL captures the distribution - including jitter and spikes. High P99 ITL means the stream occasionally stalls, which users perceive as stuttering.

E2EL - End-to-End Latency (ms)
The total time from request submission to the last token. . This is the bottom-line metric for non-streaming use cases.

How Each Parallelism Strategy Affects These Metrics

Prerequisites

* A RunPod account with GPU pod access

* A HuggingFace token (for gated models like Llama)

* Familiarity with SSH and basic Linux commands

1. RunPod Setup

1.1 Single-Node Pod (for TP, DP, single-node PP)

Provision a multi-GPU pod:

1. Navigate to Pods > Deploy.

2. Select a 4xA100 (80 GB) or 4xA6000 (48 GB) instance.

3. Pick the template (or a recent PyTorch + CUDA devel image).

4. Under Customize Deployment, expose port 8000 via HTTP (for the OpenAI-compatible server).

5. Deploy and connect via SSH or the web terminal.

1.2 Multi-Node Setup (for cross-node PP)

RunPod offers two options for multi-node:

Option A - Instant Clusters (recommended)

1. Go to Clusters in the RunPod console.

2. Create a 2-node cluster with 1xA100 (or more) per node.

3. RunPod provisions both nodes on the same private network with high-speed interconnects (InfiniBand / RoCE v2, up to 3200 Gbps).

4. You get SSH access to both nodes. Environment variables like , node IPs, and hostnames are pre-configured.

Option B - Two Separate GPU Pods (manual networking)

1. Deploy two single-GPU (or multi-GPU) pods in the same region.

2. Note down the internal/public IPs of both pods.

3. Ensure both pods can reach each other on port 6379 (Ray) and ports 8000-8100 (vLLM / NCCL). You may need to configure TCP proxy or use RunPods public IPs with exposed ports.

> *Instant Clusters are strongly preferred because they come with private network connectivity out of the box. Two separate pods communicate over the public internet, which adds latency to NCCL operations and makes cross-node TP impractical. PP is more tolerant of network latency since it only passes activations between pipeline stages.*

2. Environment Setup with `uv`

Run these steps on every node that will participate in serving.

2.1 Install `uv`

Verify:

2.2 Create a Virtual Environment and Install Dependencies

```bash
mkdir -p ~/vllm_parallelism && cd ~/vllm_parallelism

uv venv .venv python 3.11
source .venv/bin/activate

uv pip install vllm
uv pip install ray[default]
```

2.3 Verify the Setup

2.4 Set HuggingFace Token

3. Model Selection

4. Act 1 - The Prototype (Baseline, Single GPU)

You deploy your first model on a single GPU. Everything works, but you need to establish baseline metrics before optimizing.

4.1 Serve the Model on 1 GPU

Test it in a second terminal:

4.2 Measure Baseline Metrics

Run the benchmark against the running server. This reports TTFT, TPOT, ITL, and E2EL:

Flag breakdown:

* - Report percentiles for all four key metrics.

* - Report the median (P50), P90, and P99 for each metric.

* - Save the full results to a JSON file for later comparison.

* - Send 5 requests per second (simulates moderate traffic).

* - Use synthetic random prompts so the benchmark is reproducible.

Expected output format:

> *The numbers above are illustrative. Your actual numbers will depend on GPU type, model, and workload. The key is to record these as the baseline you will compare against.*

4.3 Record GPU Memory Baseline

Note the VRAM used by the model on GPU 0. All other GPUs should be idle.

5. Act 2 - The Latency Wall (Tensor Parallelism)

The Problem

Users of your coding assistant report that the model takes too long to start responding. Looking at your baseline metrics, the TTFT is 85ms at P50, but under heavier load it spikes to 145ms at P99. The prefill phase (processing the users prompt) is compute-bound - the GPU is doing all the matrix multiplications for all layers sequentially.

The Solution: Tensor Parallelism

TP shards each layers weight matrices across GPUs. Every GPU holds a slice of every layer. During the forward pass, each GPU computes its portion and the results are combined via all-reduce. This reduces per-GPU memory and can lower latency for compute-bound workloads.

5.1 TP=2 (Model Split Across 2 GPUs)

Stop the baseline server (), then:

Test it:

Benchmark:

5.2 TP=4 (Model Split Across 4 GPUs)

Test it:

Benchmark:

5.3 What to Observe

Compare the JSON result files across configurations:

```bash
python3 -c
import json

for fname, label in [
(results_baseline.json, TP=1),
(results_tp2.json, TP=2),
(results_tp4.json, TP=4),
]:
with open(fname) as f:
d = json.load(f)
print(f{label}:)
print(f  TTFT  mean={d["mean_ttft_ms"]:.1f}ms  p99={d["p99_ttft_ms"]:.1f}ms)
print(f  TPOT  mean={d["mean_tpot_ms"]:.1f}ms  p99={d["p99_tpot_ms"]:.1f}ms)
print(f  ITL   mean={d["mean_itl_ms"]:.1f}ms  p99={d["p99_itl_ms"]:.1f}ms)
print(f  Throughput: {d["output_throughput"]:.1f} tok/s)
print()

```

Expected patterns:

* TTFT drops as TP increases - the prefill computation is split across more GPUs, so each does less work.

* TPOT may slightly increase at TP=4 due to all-reduce communication overhead dominating the decode step for a relatively small model.

* ITL becomes more consistent (lower P99/P50 ratio) because compute is balanced across GPUs.

* VRAM per GPU decreases proportionally - check with .

Monitor GPU memory to confirm the split:

With TP=4, all 4 GPUs should show similar VRAM usage (roughly 1/4 of the full model each) and similar utilization.

6. Act 3 - The Memory Wall (Pipeline Parallelism)

The Problem

Product wants to upgrade from the 7B model to a 70B model for better code quality. The 70B model needs ~140 GB of VRAM in FP16. Even a single node with 4xA100-80GB (320 GB total) could fit it with TP=4, but what if you only have 2xA100-80GB? Or what if the model is even larger? You need to spread layers across multiple machines.

The Solution: Pipeline Parallelism

PP assigns different groups of consecutive layers to different GPUs (or nodes). GPU 0 runs layers 0-N/k, GPU 1 runs layers N/k+1-2N/k, and so on. Data flows sequentially through the pipeline. Unlike TP, there is

no all-reduce - only point-to-point activation transfers between pipeline stages.

6.1 Single-Node PP

PP=2 (Layers Split Across 2 GPUs)

Test it:

Benchmark:

PP=4

Test it:

Benchmark:

Combining TP + PP on a Single Node

On a 4-GPU node, you can combine both:

Test it:

Benchmark:

What to Observe

* TTFT increases slightly compared to TP - the first pipeline stage must pass activations to subsequent stages before the first token can be emitted. This is the pipeline fill latency.

* ITL can be spiky - the pipeline bubble problem means later stages idle periodically, causing uneven token emission.

* VRAM distribution is uneven - the first and last stages carry extra memory for the embedding layer and LM head. Check with .

* Throughput under high load can improve - with many requests in flight, the pipeline stages overlap computation, increasing overall throughput.

6.2 Multi-Node PP with Two RunPod Instances

This is the key section: running PP across two separate RunPod machines using Ray to form a distributed cluster.

Architecture Overview

Step 1: Provision Two RunPod Instances

Using Instant Clusters:

1. Go to Clusters in RunPod console.

2. Select 2 nodes, each with 1xA100-80GB (or more GPUs per node for TPxPP combos).

3. Deploy. RunPod assigns private IPs and sets up the interconnect.

4. SSH into both nodes. Note the private IPs:
Node 0 (head): e.g., Node 1 (worker): e.g.,

Step 2: Install Environment on Both Nodes

Run on both Node 0 and Node 1:

```bash
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env

mkdir -p ~/vllm_parallelism && cd ~/vllm_parallelism
uv venv .venv python 3.11
source .venv/bin/activate

uv pip install vllm ray[default]

export HF_TOKEN=hf_your_token_here
```

Step 3: Start the Ray Cluster

On Node 0 (head node):

```bash
export HEAD_IP=$(hostname -I | awk {print $1})
echo Head IP: $HEAD_IP

ray start head \
port=6379 \
dashboard-host=0.0.0.0 \
dashboard-port=8265
```

On Node 1 (worker node):

```bash
export HEAD_IP=10.0.0.1  # Replace with actual head node IP

ray start address=${HEAD_IP}:6379
```

Step 4: Verify the Ray Cluster

Step 5: Download the Model on Both Nodes

Step 6: Launch vLLM with Multi-Node PP

Run on the head node only:

```bash
export VLLM_HOST_IP=$(hostname -I | awk {print $1})

vllm serve Qwen/Qwen2.5-7B-Instruct \
tensor-parallel-size 1 \
pipeline-parallel-size 2 \
distributed-executor-backend ray \
```

port 8000
```

Test it:

For multi-GPU nodes (e.g., 4 GPUs per node), combine TP and PP:

Test it:

#### Step 7: Benchmark Multi-Node PP

Compare the TTFT from this run against the single-node PP=2 result. Multi-node PP will show higher TTFT and ITL due to network latency between nodes, but the model fits - which is the point.

#### Step 8: Verify Cross-Node GPU Usage

On Node 0:

On Node 1:

Each node should show roughly half the models memory footprint.

#### Troubleshooting Multi-Node

#### Cleaning Up

## 7. Act 4 - The Throughput Wall (Data Parallelism)

### The Problem

You launch your coding assistant publicly. Within a week you have 100 concurrent users. The 7B model on TP=2 handles individual requests fast (good TTFT and TPOT), but under high concurrency, requests queue up. The servers throughput caps at ~300 tok/s, and P99 TTFT spikes to 500ms+ because new requests wait for in-flight requests to free up KV cache slots.

### The Solution: Data Parallelism

DP replicates the entire model on each GPU (or group of GPUs) and splits incoming requests across replicas. Each replica independently processes its share of requests. No gradient synchronization is needed (this is inference, not training), so DP scales throughput almost linearly with the number of replicas.

### 7.1 DP=2 (Two Model Replicas)

Test it:

### 7.2 DP=2 x TP=2 (Four GPUs Total)

Test it:

### 7.3 Benchmarking DP Under Load

DP shines under high concurrency. The key is to increase  and  to simulate realistic multi-user traffic:
```

```bash
# Benchmark with DP=1 (baseline under heavy load)
vllm serve $MODEL tensor-parallel-size 1 port 8000 &
sleep 30

vllm bench serve \
model $MODEL \
base-url http://localhost:8000 \
backend openai-chat \
dataset-name random \
random-input-len 512 \
random-output-len 128 \
num-prompts 200 \
request-rate 20 \
max-concurrency 50 \
percentile-metrics ttft,tpot,itl,e2el \
metric-percentiles 50,90,99 \
save-result \
output-json results_dp1_highload.json

kill %1
sleep 5
```

```bash
# Benchmark with DP=2 under the same heavy load
vllm serve $MODEL data-parallel-size 2 tensor-parallel-size 1 port 8000 &
sleep 30

vllm bench serve \
model $MODEL \
base-url http://localhost:8000 \
backend openai-chat \
dataset-name random \
random-input-len 512 \
random-output-len 128 \
num-prompts 200 \
request-rate 20 \
max-concurrency 50 \
percentile-metrics ttft,tpot,itl,e2el \
metric-percentiles 50,90,99 \
save-result \
output-json results_dp2_highload.json

kill %1
```

7.4 What to Observe

Compare the two JSON files:

```bash
python3 -c
import json

for fname, label in [
(results_dp1_highload.json, DP=1 (high load)),
(results_dp2_highload.json, DP=2 (high load)),
]:
with open(fname) as f:
d = json.load(f)
print(f{label}:)
print(f  Throughput: {d["output_throughput"]:.1f} tok/s)
print(f  TTFT  p50={d["median_ttft_ms"]:.1f}ms  p99={d["p99_ttft_ms"]:.1f}ms)
print(f  TPOT  p50={d["median_tpot_ms"]:.1f}ms  p99={d["p99_tpot_ms"]:.1f}ms)
print(f  ITL   p50={d["median_itl_ms"]:.1f}ms  p99={d["p99_itl_ms"]:.1f}ms)
print(f  E2EL  p50={d["median_e2el_ms"]:.1f}ms  p99={d["p99_e2el_ms"]:.1f}ms)
print()

```

Expected patterns:

* Throughput roughly doubles from DP=1 to DP=2 (two independent replicas processing requests in parallel).

* P99 TTFT drops significantly - with DP=2, requests are distributed across replicas, so queue wait times shrink.

* TPOT stays similar per request (each replica runs the same model independently).

* P99 ITL drops - less contention means fewer scheduling stalls.

* VRAM: Each GPU loads the full model. Total VRAM usage is .

8. Act 5 - Production (Putting It All Together)

In production, you combine strategies based on your constraints:

The constraint: TP x PP x DP = Total GPU count

For example, on a 2-node cluster with 4 GPUs per node (8 GPUs total):

* TP=4 x PP=2 x DP=1 = 8 GPUs - One large-model replica across 2 nodes

* TP=2 x PP=1 x DP=4 = 8 GPUs - Four small-model replicas, each using TP=2

* TP=2 x PP=2 x DP=2 = 8 GPUs - Two replicas, each spanning 2 nodes with TP=2

9. Automated Benchmark Sweep

Create a script to run all single-node configurations and collect comparable metrics:

```bash
#!/bin/bash
# benchmark_sweep.sh
# Run from the single-node 4-GPU pod

MODEL=Qwen/Qwen2.5-7B-Instruct
BENCH_ARGS=backend openai-chat dataset-name random random-input-len 512 random-output-len 128 num-prompts 100 request-rate 5 percentile-metrics ttft,tpot,itl,e2el metric-percentiles 50,90,99 save-result

run_config() {
local label=$1
local output_file=$2
shift 2
local serve_args=$@

echo "=========================================="
echo ">>> Starting: $label"
echo ">>> Serve args: $serve_args"
echo "=========================================="
vllm serve $MODEL $serve_args --port 8000 &
local pid=$!
echo "Waiting for server..."
for i in $(seq 1 60); do
if curl -s http://localhost:8000/v1/models > /dev/null 2>&1; then
echo "Server ready after ${i}s"
break
fi
sleep 1
done
vllm bench serve \
--model $MODEL \
--base-url http://localhost:8000 \
$BENCH_ARGS \
--output-json "$output_file"
kill $pid 2>/dev/null
wait $pid 2>/dev/null
sleep 5
echo "" }
```

```
run_config Baseline (TP=1) sweep_tp1.json tensor-parallel-size 1
run_config TP=2 sweep_tp2.json tensor-parallel-size 2
run_config TP=4 sweep_tp4.json tensor-parallel-size 4
run_config PP=2 sweep_pp2.json pipeline-parallel-size 2
run_config PP=4 sweep_pp4.json pipeline-parallel-size 4
run_config TP=2 PP=2 sweep_tp2_pp2.json tensor-parallel-size 2 pipeline-parallel-size 2
run_config DP=2 sweep_dp2.json data-parallel-size 2 tensor-parallel-size 1
run_config DP=2 TP=2 sweep_dp2_tp2.json data-parallel-size 2 tensor-parallel-size 2

echo
echo ==========================================
echo All benchmarks complete. Summary:
echo ==========================================

python3 PYEOF
import json, glob, os

files = sorted(glob.glob(sweep_*.json))
header = f{Config:<16} {Tput(tok/s):>12} {TTFT p50:>10} {TTFT p99:>10} {TPOT p50:>10} {TPOT p99:>10}
{ITL p99:>10} {E2EL p99:>10}
print(header)
print(- * len(header))

for f in files:
label = os.path.basename(f).replace(sweep_, ).replace(.json, ).upper()
with open(f) as fh:
d = json.load(fh)
print(f{label:<16} {d[output_throughput]:>10.1f}/s {d[median_ttft_ms]:>8.1f}ms {d[p99_ttft_ms]:>8.1f}ms
{d[median_tpot_ms]:>8.1f}ms {d[p99_tpot_ms]:>8.1f}ms {d[p99_itl_ms]:>8.1f}ms
{d[p99_e2el_ms]:>8.1f}ms)
PYEOF
```

Expected Results Pattern

*Under high concurrency. At low concurrency, DP shows no improvement since there is no queueing.

10. Prometheus Metrics (Live Monitoring)

Beyond benchmarking, vLLM exposes real-time Prometheus metrics on the endpoint. You can scrape these while the server is running:

Key metrics available:

* - Histogram of TTFT values
* - Histogram of ITL values
* - Histogram of end-to-end latency
* - Time spent in prefill phase

* - Time spent in decode phase

* - Current in-flight requests

* - Requests queued (high values here signal the need for DP)

11. GPU Monitoring Cheat Sheet

```bash
# Continuous monitoring
watch -n 1 nvidia-smi

# Compact CSV output
nvidia-smi query-gpu=index,name,memory.used,memory.total,utilization.gpu,temperature.gpu \
format=csv -l 1

# Process-level GPU usage
nvidia-smi pmon -s um -c 1
```

What to look for per configuration:

* TP: All GPUs show similar VRAM usage and similar GPU utilization.

* PP: GPUs show different VRAM amounts (first/last stage slightly higher due to embedding/LM head). Utilization may be uneven due to pipeline bubbles.

* DP: Each GPU shows full model VRAM. All GPUs at high utilization under load.

12. Quick Reference

vLLM Serve Flags

vLLM Bench Serve Flags (Metrics Collection)

13. Cleanup

If using RunPod Instant Clusters, terminate the cluster from the RunPod console when done to stop billing.

Summary: The Story Arc

1. Prototype - Single GPU, baseline metrics recorded. TTFT is okay, throughput is limited.

2. Latency Wall - TP splits layers across GPUs. TTFT drops. Users see faster first responses.

3. Memory Wall - PP splits layer groups across nodes. Models that cannot fit on one machine now run.

4. Throughput Wall - DP replicates the model. Throughput scales linearly. P99 latencies drop under load.

5. Production - Combine TP x PP x DP to serve large models at scale with acceptable latency and throughput.

Each strategy addresses a different bottleneck. Measuring TTFT, TPOT, ITL, and E2EL at every step tells

you exactly which wall you have hit and whether the strategy you applied actually moved the needle.